

Low-Level HW Digital Systems II

Lab Coursework 2026

Gkentzi Alexandra

Contents

1	Project Description	3
2	Module Analysis.....	3
2.1	Exponent Calculation Module.....	3
2.2	Mantissa Calculation Module.....	3
2.3	Normalization Module	4
2.4	Rounding Module.....	4
2.5	Exception Handling Module	5
3	Testbench Implementation	6
4	System Verilog Assertions (SVA)	7
4.1	Immediate Assertions: Status bits.....	7
4.2	Concurrent Assertions: Status – Result Combinations.....	8
5	Conclusion & Waveforms.....	8

1 Project Description

This coursework focuses on the design and verification of a 32-bit, single-precision floating-point adder compliant with the IEEE-754 standard using System Verilog. The objective of the project is to develop a modular floating-point arithmetic unit capable of performing addition operations between two 32-bit floating-point numbers while handling normalization, rounding, overflow/underflow conditions, and special corner cases.

The implementation is divided into multiple modules, including exponent calculation, mantissa alignment and calculation, normalization, rounding, exception handling, and a leading-zero counter. A top-level wrapper module is also provided to integrate the complete design.

In addition to the hardware implementation, the project requires the development of a comprehensive verification environment. A System Verilog testbench is used to validate the functionality of the floating-point adder through both randomized and corner-case testing. The produced results are compared against a reference model based on the HardFloat library.

Furthermore, System Verilog Assertions (SVA) are implemented to verify the correctness of the generated status flags and to ensure proper operation of the design under different conditions.

2 Module Analysis

2.1 Exponent Calculation Module

The **exponent_calc.sv** file contains the implementation of the exponent calculation module. This module takes two 8-bit input signals (**exp_a** and **exp_b**), which correspond to the exponents of two 32-bit single-precision floating-point numbers. Its primary purpose is to compare the two exponents and store the larger value in the **max_exp** output. Furthermore, it calculates their absolute difference and assigns it to the 9-bit **exp_diff** output in order to prevent overflow errors. Finally, it asserts the **sign_exp_diff** signal (sets it to 1) if **exp_a** is strictly greater than **exp_b**, otherwise it remains 0. This procedure is known as exponent alignment and is essential during floating-point addition and subtraction.

2.2 Mantissa Calculation Module

The **mant_calc.sv** file contains the module for aligning and adding or subtracting the mantissas. It takes as inputs two 24-bit mantissas (**mant_a** and **mant_b**), the exponent difference (**exp_diff**), the sign of this difference (**sign_exp_diff**), and the sign bits of the original 32-bit numbers. After the calculations, the final 28-bit mantissa is produced as the output.

In the first **always_comb** block, the alignment is done. The **sign_exp_diff** signal determines the **large_mant** depending on which number has the bigger exponent. In case the exponents are equal, the selection is based on which number has the bigger mantissa. If the exponent difference is greater than 48, all the bits of the **small_mant** will be lost during the right shift, so only the sticky bit is kept at 1. In any other case, to make sure no bits are lost during the shift and the GRS bits are calculated correctly, 25 zeros are added to the right of the number.

Then, the mantissa with the smaller exponent is shifted to the right, and the final sticky bit is found using an OR operation on the 23 LSBs. Also, 3 bits are added to the right of the **final_large_mant** so it matches the size of the **final_small_mant**. Additionally, a leading zero is added to the MSB position of both **final_large_mant** and **final_small_mant** to prevent overflow during the addition or subtraction in the next block.

Next, in the second **always_comb** block, the addition or subtraction of the two aligned mantissas takes place, and the result is stored in the 28-bit **result_mant** output. If the signs of the two original numbers are the same, an addition is performed, otherwise, a subtraction takes place.

2.3 Normalization Module

Since the result of the mantissa addition needs to be in the 1.xxxxx format, it must be normalized. Therefore, the normalization module, located in the **norm_adder.sv** file, takes the result from the mantissa calculation (**result_mant**) and the larger exponent of the two numbers as inputs. It outputs the normalized 9-bit exponent and the normalized 27-bit mantissa.

To support this process, an additional module is implemented in the **lzc.sv** file. This module counts the number of zeros before the first '1' appears on the left side of the mantissa result. Basically, a for-loop checks the bits from left to right until a '1' is found. If no '1' is detected at all, the **leading_zeroes** variable defaults to 28, which matches the total number of bits.

If the mantissas result is greater than or equal to 2 (meaning it has a 1x.xxx format), means that the MSB is 1. In this case a Logical Shift Right (**LSR**) is applied to **result_mant**. To avoid losing precision, the last bit is not simply discarded, instead a logical OR operation is performed between the last two bits to create the new sticky bit. Since the mantissa was shifted right, the exponent must be increased by 1. To prevent overflow during this addition, the exponent is first extended to 9 bits by adding a 0 to its left side. If only the second MSB is 1, the leading zero MSB is simply dropped, and the exponent is just extended to 9 bits. Furthermore, if the result is zero, both the mantissa and exponent are directly set to zero. Finally, if the result is less than 1, a Logical Shift Left (**LSL**) by **leading_zeroes - 1** is performed. If the exponent is smaller than **leading_zeroes - 1**, subtracting them would cause an underflow because the exponent is an unsigned number. Therefore, the final if-else block verifies this condition and either zeroes out the exponent, or safely subtracts **leading_zeroes - 1** and assigns the result to **norm_exp**.

2.4 Rounding Module

The final stage of a floating-point operation is rounding. Since there are various rounding methods, the **round_pkg.sv** file is used to define an enum that assigns a 3-bit value to each method.

The **round_adder** module, located in the **round_adder.sv** file, takes as inputs the rounding method (**round**), the normalized 27-bit mantissa (**norm_mant**), and the sign of the final result (**z_sign**). It outputs the 25-bit rounded mantissa, which includes an extra bit for potential overflow, and the **inexact_bit**, which indicates if any precision was lost.

Inside the **always_comb** block, the first step is to perform a logical OR operation between the **G** (guard), **R** (round), and **S** (sticky) bits and store the result in the **inexact_bit**. Next, a case statement is used to separate the necessary operations based on the selected rounding mode. The process for each method is explained below:

- **IEEE_near**: Round to nearest, ties to even. If the guard bit is 1 and at least one of the round or sticky bits is 1, the value is closer to the next number, so 1 is added. However, if **R = S = 0**, there is a tie. In this case, the first 23 bits are kept and **norm_mant[3]** is added. If **norm_mant[3]** is 1 the number is odd so adding it creates an even number. If it is 0, the number remains even. In any other scenario, the GRS bits are simply dropped.
- **IEEE_zero**: Round to zero. The GRS bits are simply dropped.
- **IEEE_ninf**: Round to negative infinity. The initial number is checked to see if it is negative (**z_sign = 1**) and if any information was lost (**inexact_bit = 1**). If both conditions are true, 1 is added to the mantissa to increase its absolute magnitude. In any other case, the GRS bits are simply dropped.
- **IEEE_pinf**: Round to positive infinity. It checks if the number is positive (**z_sign = 0**) and if there is a loss of precision (**inexact_bit = 1**). If so, 1 is added to the mantissa to increase its value. If the number is negative or no information was lost, the last 3 bits are simply dropped.
- **near_maxMag**: Round to nearest, ties to maximum magnitude. If the guard bit is 1, 1 is simply added to the mantissa. If the guard bit is 0, the last 3 bits are simply dropped.

2.5 Exception Handling Module

Finally, the **exception_adder.sv** file was created, which contains the **exception_adder** module and is responsible for handling all the corner cases. Specifically, two functions were implemented. First, the **num_interp** function was implemented, which checks if the exponent of the 32-bit number is 8'b1111_1111, 8'b0000_0000, 8'b0000_0001, 8'b1111_1110 or anything else, and returns **INF**, **ZERO**, **MIN_NORM**, **MAX_NORM** or **NORM** respectively. This way, it identifies whether the inputs fall under the special values of the IEEE 754 standard. The second function, which is **z_num**, returns the 31-bit value corresponding to the boundary limits of **ZERO**, **INF**, minimum normal number (**MIN_NORM**), and maximum normal number (**MAX_NORM**). Next, an **always_comb** block was created, where first, all exception flags are initialized to zero to prevent the creation of latches. Then, inside the block, the following cases are handled:

1. If both inputs are zero, then the result will be zero, and **zero_f** is activated. If the inputs have the same sign, that sign is given to the result. However, if they have different signs, the sign of the result must be positive (0), unless the rounding mode is **IEEE_ninf**, then the sign must be negative (1). The **zero_sign** signal is implemented to cover this case.
2. If at least one of the inputs is **INF**, the module checks if both are **INF** but with different signs. According to IEEE 754 standard this operation is invalid and the result is NaN, which is why **nan_f** is activated. However, based on the specific

assignment instructions, the result must be treated as infinity, that's why the result is assigned the value 32'h7F80_0000. In any other case, the result will be **INF** with the appropriate sign, and **inf_f** is activated.

3. If the overflow signal is active, then **huge_f** and **inexact_f** are activated, and depending on the rounding method and the sign, it is selected whether the result will become **INF** or the maximum normal number (**MAX_NORM**). If the rounded result becomes INF, the **inf_f** flag is also activated.
4. If the underflow signal is active, the same procedure as overflow is followed, but it activates **tiny_f** and **inexact_f**, and the result will be either **ZERO** or the minimum normal number (**MIN_NORM**). If the rounded result becomes ZERO, the **zero_f** flag is also activated.
5. Finally, if none of the above exceptions occur, it passes the initial result to the output, updates **inexact_f** depending on the **inexact_bit**, and checks if the final result equals zero to activate **zero_f**.

3 Testbench Implementation

To verify the correct operation of the circuit, the **testbench_adder.sv** file was implemented, based on the provided **tb_adder.sv** file. The testbench compares the results against a reliable reference model.

The clock period was set to 10 ns, as requested by the assignment.

To generate the expected correct results, a ready-made code block is used that utilizes modules from the Hardfloat library. Since this library operates with 33-bit numbers, it was necessary to use the ready-made modules (**fNToRecFN**, **recFNToFN**) to convert the inputs a and b from 32 to 33 bits, and the result back from 33 to 32 bits, so that the comparison could be made.

Subsequently, two tasks were created, one for testing random combinations and one for testing corner cases. In the first task, 10000 random input combinations (a and b) were generated using the **\$urandom()** function. Each pair was tested across all 5 possible rounding modes, and if the result differed from the reference model, an error message was printed. Additionally, an error variable (**random_error**) counted how many tests failed, while **random_total** counted how many tests were executed.

In the second task, the 100 corner cases were tested for each rounding mode as well. For convenience, the **corner_pkg.sv** file was created, declaring the **corner_t** enum that mapped all the corner cases, along with the **corner_val** function, which returns the corresponding value for each case. Therefore, a nested loop implementation was used to test each combination exactly once. Similarly, this task also prints failure messages and counts how many errors occurred and how many tests were executed.

Then, in an initial block, after initializing the inputs and the rounding mode to 0, the two tasks are called. Furthermore, at the end of all tests, it prints a final message that show how many tests passed out of the total number of tests performed for every task.

Since the circuit requires 2 clock cycles to calculate the final result but accepts a new input pair every clock cycle, the comparisons had to be delayed accordingly, because the reference model produced results immediately. To implement this, an **always_ff** block and two variables, **exp_r1** and **exp_r2**, were required. In the first cycle, the correct result is stored in **exp_r1**, while in the second cycle, the correct result is passed to **exp_r2**. This ensured that **exp_r2** holds the expected result, delayed by exactly two cycles. To ensure the comparison only begins after this initial two-cycle delay, the **random_total** and **corner_total** variables were utilized as counters. Finally, since the comparison begins two cycles after the initial input insertion, it must also conclude 2 cycles after the inputs stop. For this reason, the process is repeated 2 times after the main loop inside the tasks.

4 System Verilog Assertions (SVA)

System Verilog Assertions (SVAs) are an essential part of verifying the correct operation of the circuit. For this assignment, the implementation of two modules was required. To continuously monitor these signals, SVA modules were bound directly to the **fp_adder_top** wrapper module.

4.1 Immediate Assertions: Status bits

The first module, in **test_status_bits.sv** file, is responsible for checking the flags that comprise the status variable. Specifically, it verifies whether certain flags are activated simultaneously when they should not be. According to Table 3, the impossible combinations are the following:

Impossible Combinations	
Zero	Infinity
Zero	Not a Number
Infinity	Not a Number
Tiny	Huge
Tiny	Infinity
Tiny	Not a Number
Huge	Zero
Huge	Not a Number
Inexact	Not a Number

Therefore, 9 Immediate Assertions were implemented inside an **always_comb** block. Each assertion prints a PASS message using **\$display** when the condition is satisfied, and a FAIL message using **\$error** when the condition is violated. The printed message also includes the simulation time and the assertion hierarchy, making it easier to identify which assertion was evaluated. Additionally, an **if(resetn)** condition was added at the beginning of the module to prevent the assertions from being evaluated before the circuit has been reset, when the status variable may still contain unknown values.

4.2 Concurrent Assertions: Status – Result Combinations

In the second module, located in the `test_status_z_combinations.sv` file, the following four conditions are checked using Concurrent Assertions:

1. If the 'zero' status bit asserts to 1, then at the same cycle all the bits of the exponent of 'z' must be equal to 0.
2. If the 'inf' status bit asserts to 1, then at the same cycle all the bits of the exponent of 'z' must be equal to 1.
3. If the 'nan' status bit asserts to 1, then 2 cycles before all the bits of the exponent of 'a' and 'b' must be equal to 1, and the 'a' and 'b' signals must have opposite signs.
4. If the 'huge' status bit asserts to 1, then at the same cycle all the bits of the exponent of 'z' must be equal to 1, or all the bits of the exponent of 'z' except the LSB must be equal to 1, the LSB must be 0, and all the bits of the mantissa of 'z' must be equal to 1 (maxNormal case).

Each rule was implemented as a separate property, which is evaluated at every positive clock edge to remain synchronized with the circuit's data flow. Furthermore, the overlapping implication operator (`|>`) was used. Based on this operator, if the condition on the left side is true, the condition on the right side must also be true during the same clock cycle. To execute the check, the assert property statement is used. When the property is satisfied, a PASS message is printed using `$display`, while if the property fails, a FAIL message is printed using `$error`.

5 Conclusion & Waveforms

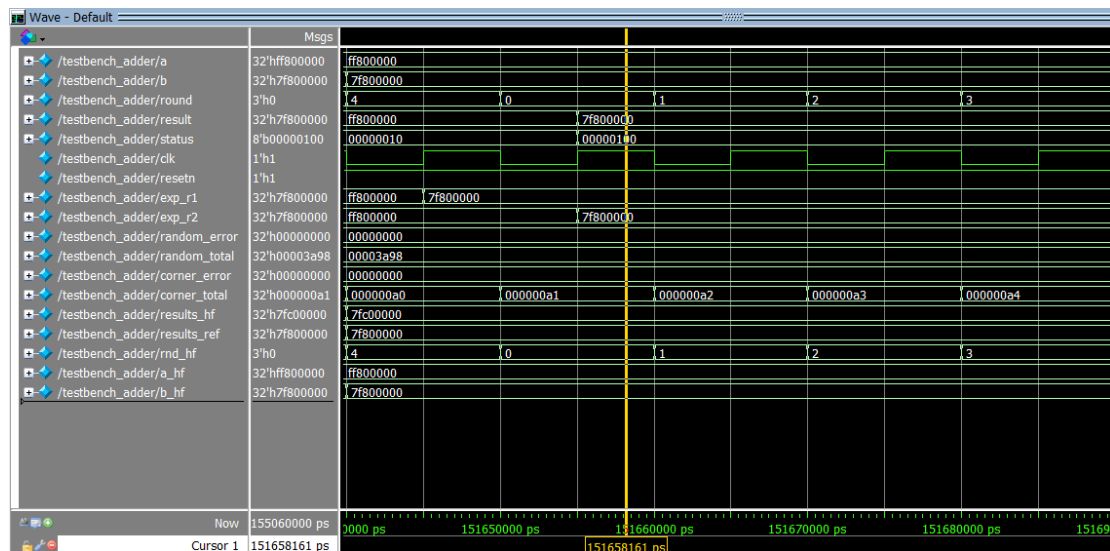
In conclusion, the design and implementation of the Floating Point Adder were completed with absolute success. The circuit passed the 50000 random tests and 500 corner cases without a single error, while also fully satisfying all the SVA checks.

Final Test Summary in Questa Transcript

```
# Executed Tests 50500
# Random Tests SUCCESS 50000 / 50000
# Corner Tests SUCCESS 500 / 500
# ** Note: $finish      : C:/Users/Alexandra/Documents/HMMY/HW2/testbench_adder.sv(207)
#    Time: 505060 ns  Iteration: 1  Instance: /testbench adder
```

The screenshot above shows the final message printed at the end of the simulation, indicating the total number of executed tests and the number of successful tests for both random and corner cases.

Waveform Comparison Between DUT and Reference Model Outputs



The diagram above confirms the correct operation of the 2-cycle pipeline. The internal latency of the circuit is exactly 2 clock cycles, meaning it requires two positive clock edges to process the data. However, because the new inputs are applied on the negative clock edge to avoid race conditions, the final result appears to be generated after two positive edges, resulting in a visual delay of 1.5 clock cycles. This observed behavior perfectly aligns with the delayed golden reference model (**exp_r2**), ensuring accurate verification.

Furthermore, it shows a specific corner case, the addition of negative and positive infinity ($a = \text{ff800000}$ and $b = \text{7f800000}$). The mathematical result of this operation is a NaN (Not a Number) but according to the assignment specifications, NaN values must be handled as infinity. This behavior is verified in the waveforms, as the result variable takes the value of positive infinity (7f800000), while the NaN flag is correctly asserted (**status[2] = 1**).

Assertion PASS Messages in the Questa Transcript

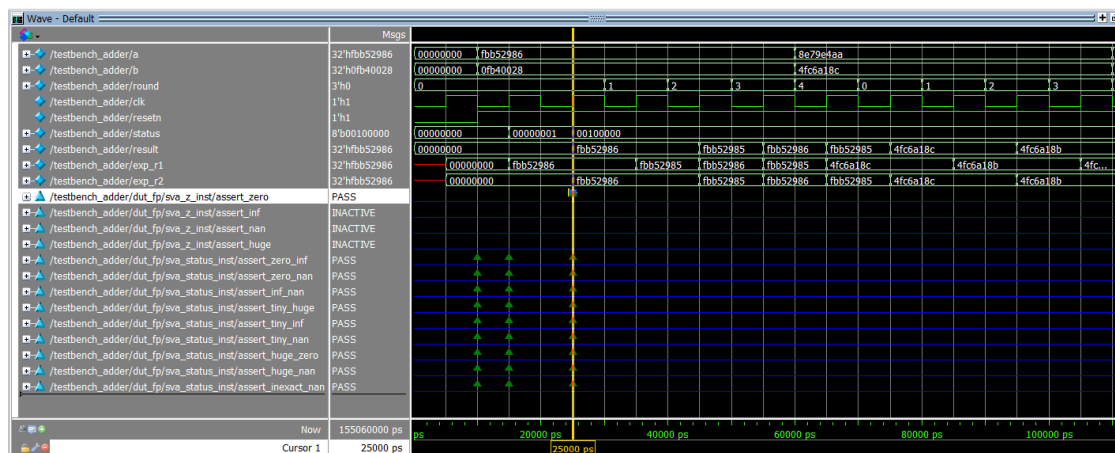
```
# 256665000 testbench_adder.dut_fp.sva_z_inst.assert_inf PASS
# 256665000 testbench_adder.dut_fp.sva_z_inst.assert_huge PASS
# 256675000 testbench_adder.dut_fp.sva_status_inst.assert_zero_inf PASS
# 256675000 testbench_adder.dut_fp.sva_status_inst.assert_zero_nan PASS
# 256675000 testbench_adder.dut_fp.sva_status_inst.assert_inf_nan PASS
# 256675000 testbench_adder.dut_fp.sva_status_inst.assert_tiny_huge PASS
# 256675000 testbench_adder.dut_fp.sva_status_inst.assert_tiny_inf PASS
# 256675000 testbench_adder.dut_fp.sva_status_inst.assert_tiny_nan PASS
# 256675000 testbench_adder.dut_fp.sva_status_inst.assert_huge_zero PASS
# 256675000 testbench_adder.dut_fp.sva_status_inst.assert_huge_nan PASS
# 256675000 testbench_adder.dut_fp.sva_status_inst.assert_inexact_nan PASS
```

This screenshot shows the PASS messages printed in the Questa Transcript after the successful evaluation of several assertions, during the time 256665 ns to 256675 ns.

Verification Results of Immediate and Concurrent Assertions

Name	Assertion Type	Language	Enable	Failure Count	Pass Count
/testbench_adder/dut_fp/sva_z_inst/assert_zero	Concurrent	SVA	on	0	1
/testbench_adder/dut_fp/sva_z_inst/assert_inf	Concurrent	SVA	on	0	1
/testbench_adder/dut_fp/sva_z_inst/assert_nan	Concurrent	SVA	on	0	1
/testbench_adder/dut_fp/sva_z_inst/assert_huge	Concurrent	SVA	on	0	1
/testbench_adder/dut_fp/sva_status_inst/assert_zero_inf	Immediate	SVA	on	0	1
/testbench_adder/dut_fp/sva_status_inst/assert_zero_nan	Immediate	SVA	on	0	1
/testbench_adder/dut_fp/sva_status_inst/assert_inf_nan	Immediate	SVA	on	0	1
/testbench_adder/dut_fp/sva_status_inst/assert_tiny_huge	Immediate	SVA	on	0	1
/testbench_adder/dut_fp/sva_status_inst/assert_tiny_inf	Immediate	SVA	on	0	1
/testbench_adder/dut_fp/sva_status_inst/assert_tiny_nan	Immediate	SVA	on	0	1
/testbench_adder/dut_fp/sva_status_inst/assert_huge_zero	Immediate	SVA	on	0	1
/testbench_adder/dut_fp/sva_status_inst/assert_huge_nan	Immediate	SVA	on	0	1
/testbench_adder/dut_fp/sva_status_inst/assert_inexact_nan	Immediate	SVA	on	0	1

Waveform View with Assertion Pass Markers



The two images above visually confirm that the SVAs are working exactly as intended. In the Verification Results window, a Pass Count of 1 means that the corresponding assertion passed at least once, while a Failure Count of 0 means that no failure was detected for that assertion. Specifically, at 25,000 ps, we observe the zero flag being asserted (`status[0] = 1`), which triggers `assert_zero`, passing the check successfully. At the same time, the immediate assertions appear to run whenever any bit of the status changes, confirming that no forbidden flag combinations occurred. Also, the waveform viewer displays the assertion markers, where the green triangles indicate successful assertion evaluations, while failures would be marked accordingly.